



evg_zlg

54 минуты назад

Как мы делали фасетный поиск для большого каталога: от «на коленке» к InstantSearch.js и к TanStack Query



Средний



5 мин



1.2K

Разработка под e-commerce*, ReactJS*

Ретроспектива

Фасетный поиск в eCommerce — штука коварная. Пока фильтров три и категорий пять — можно написать примерно любое своё решение и оно будет работать. Но когда каталог растёт, появляются десятки фасетов, динамические атрибуты, а пользователи начинают кликать по фильтрам быстрее, чем успевает обновляться интерфейс, — тут-то и начинаются сложности.

В этой статье я расскажу, как мы прошли путь от самописного велосипеда через InstantSearch.js с кастомным клиентом до связки TanStack Query + nuqs. С костылями, сомнениями и парой архитектурных «а что, если...».

Важно: это описание нашего практического опыта, а не истина в последней инстанции. Возможно, мы что-то делали не так, где-то не докрутили, а где-то луна была не в той фазе. Если у вас получилось подружить InstantSearch с кастомным поисковым движком без лишних проблем — мы только за. Нам же хочется поделиться тем, к чему пришли сами, и, возможно, сэкономять кому-то время.

Контекст

B2C eCommerce-платформа с большим каталогом продуктов. Гибридный SSR/CSR, SEO важен, пользователи активно используют фильтры.

Составляющие

- **Каталог** — 40+ миллионов SKU, вариативные товары.
- **Поисковый движок** — Manticore Search.
- **Фронтенд** — Next.js, UI-компоненты на базе shadcn/ui.
- **UI каталога** — фильтры (фасеты), сортировка, пагинация, синхронизация состояния с URL.

Особенности

- **15–20+ фасетов** на категорию, плюс динамические атрибуты, которые PIM-система может менять без участия разработчиков.

- **Disjunctive faceting** — логика «ИЛИ» внутри фасета (например, несколько брендов) и «И» между разными фасетами.
 - **SSR обязателен** — гидратация без мерцаний и двойных запросов.
 - **Быстрый отклик** и предсказуемая нагрузка на поисковый движок.
-

Этап 1. Самописная реализация «на коленке»

На старте было просто: пара фильтров, прямые запросы в Manticore, состояние в `useContext`, URL-синхронизация руками.

Что пошло не так:

- Каждый новый фильтр требовал правки в нескольких местах: запросы, агрегации, UI.
- Disjunctive faceting превращал код в «лапшу» из ветвлений.
- Постоянное дублирование логики между SSR и CSR, плюс мерцания при гидратации.

Прорыв (который нас и подтолкнул к InstantSearch):

Мы поняли, что фильтры должны быть динамическими — их набор и возможные значения определяются PIM и могут меняться без переписывания фронтенда. Ручное добавление каждого фасета перестало масштабироваться. Статические фильтры тоже остаются (цена, наличие), но всё остальное можно передать в динамику.

Этап 2. InstantSearch.js с кастомным клиентом (без Algolia)

Звучит логично: есть библиотека, которая умеет всё — фасеты, состояние, синхронизацию с URL. Берём, адаптируем под Manticore, получаем профит.

Спойлер: есть нюансы.

С чем мы столкнулись

- **Двойной запрос при SSR.** Server action для SSR + API handler для клиента. Из-за клиентской природы InstantSearch вызов API не мог идти через server action.
- **Каждый смонтированный фильтр = отдельный запрос.** Кеш спасал, но рассинхрон был возможен.
- **Документации по кастомному searchClient нет.** Формат данных искали методом тыка.
- **InstantSearch превратился из помощника в препятствие.**
 - Появились вынужденные прослойки вроде `FiltersController`.
 - В компонентах постоянно приходилось сверяться: что изменилось, что в URL, первый это рендер или нет.

- **Отдельный квест — первый клиентский переход.** Если сначала открыть главную страницу, а затем перейти на страницу с фильтрами — поведение отличалось от прямой загрузки (SSR).

- **Гонка состояний.** Пользователь кликает быстро — результат непредсказуем. Пришлось блокировать интерфейс на время загрузки.

Сомнения и вывод

В процессе реализации то и дело возникал вопрос: «А может, мы выбрали не тот путь?» Ведь библиотека написана умными людьми, потрачено много усилий — скорее всего, это мы что-то делаем не так.

Сейчас мы считаем, что `InstantSearch` — отличное решение, **когда вы работаете в экосистеме Algolia**. Там он даёт простоту, хороший SSR и предсказуемое поведение. Но как только возникает необходимость в кастомном `searchClient` (свой поисковый движок, нестандартная бизнес-логика), выгода от использования `InstantSearch` стремительно тает, а цена поддержки растёт.

Сроки: около трёх месяцев до стабильной версии (с костылями, но работающей).

Этап 3. TanStack Query как слой серверного состояния

Когда стало понятно, что `InstantSearch` не наш путь, мы решили: оставляем API как есть, но выкидываем `InstantSearch`. Всё, что связано с кэшированием, дедупликацией и передачей состояния с сервера на клиент, отдаём TanStack Query. URL-синхронизацию — отдельному хуку на `nuqs`.

Ключевые отличия от `InstantSearch`:

- Нет API handler — всё через server action.
- Нет `FiltersController` и стейт-менеджеров — состояние фильтров живёт в URL и `queryKey`.
- При изменении фильтра — один запрос, без дублей.

Любое изменение фильтров меняет `queryKey`, и TanStack Query сам решает, отдать данные из кэша или сходить за новыми.

Что мы получили

- **Контроль и предсказуемость.** Запросы, агрегации, инвалидация — всё явно, нет чёрных ящиков.
- **Меньше сетевых вызовов.** Кэш + дедупликация.
- **Стабильный SSR.** Один запрос на сервере, гидратация без повторного fetch.
- **Исчезла блокировка интерфейса.** Можно кликать с любой скоростью — пользователь всегда увидит результат последнего изменения.
- **Проще развивать.** Добавление нового статического фильтра — расширить `queryKey` и UI-компонент. Без адаптеров и костылей.

- **Оптимистичные обновления.** Гулять так гулять. Пользователь видит изменение мгновенно, а запрос идёт в фоне. При быстрых кликах интерфейс не подвисает.

Сроки: около двух недель на переход. Почти все наработки по API и динамическим фильтрам переиспользовали.

Минусы TanStack Query

- **Порог входа выше, чем у InstantSearch + Algolia.** Но он понятный и предсказуемый. Всё ок с документацией.
- **Больше ручной работы.** Нет готовых UI компонентов. В нашем случае UI уже был кастомным, так что не заметили.
- **Риск раздуть кэш** при неаккуратных ключах.
- **SSR-гидратация всё равно требует внимания к архитектуре.** TanStack Query решает проблемы данных, но не роутинг и не синхронизацию с URL.

Резюме

В итоге мы получили предсказуемое, быстрое и понятное решение. Основные изменения:

- **Кода стало меньше** — исчез слой адаптера под Algolia-контракты и `FiltersController`.
- **Состояние фильтров живёт в URL** и нормализовано в `queryKey`.
- **SSR работает без костылей** — один запрос на сервере, гидратация без повторного fetch.
- **Дедупликация и кэш** снизили нагрузку на движок.

Что выбрать?

- **InstantSearch.js** — отличное решение, **если вы работаете в экосистеме Algolia**. Быстро, просто, с хорошим (возможно) SSR из коробки.
- **TanStack Query + nuqs** — когда у вас кастомный поисковый движок, нестандартная логика фасетов или вы не хотите привязываться к Algolia. Даёт контроль и прозрачность, но требует больше ручной работы.

Наш опыт показывает: для кастомного бэкенда связка TanStack Query + nuqs оказалась проще, быстрее и дешевле в поддержке, чем попытки подружить InstantSearch с Manticore.

Теги: [Nextjs](#), [manticore search](#), [algolia search](#), [InstantSearch](#), [facets](#)

Хабы: [Разработка под e-commerce](#), [ReactJS](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться



5

Карма

@evg_zlg

Пользователь

Подписаться



Поток Фронтенд доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для фронтендеров

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

[Перейти в поток Фронтенд](#)

[Комментировать](#)

Публикации

[ЛУЧШИЕ ЗА СУТКИ](#)

[ПОХОЖИЕ](#)



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой

4 мин

16K

+96

30

27



interpres

18 часов назад

Электронинструмент становится хуже, и это делается намеренно

Простой

11 мин

26K

Обзор

Перевод

+68

45

81



prohronus

18 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

5 мин

9.2K

Кейс

+51

9

9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

Простой

7 мин

12K

Обзор

+37

59

18



beget_com

23 часа назад

Механический калькулятор. Как работает арифмометр?

Средний

10 мин

9.5K

Обзор

+37

23

8



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

5 мин

8.7K

Обзор

+27

10

0



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

Средний

14 мин

9.9K

Мнение

+26

33

11



mariklolik

18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

Средний

7 мин

7.4K

Кейс

+23

23

6



nick_dyuba

22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

5 мин

7K

Recovery Mode

+22

8

5



Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

11 мин

6.9K

+17

9

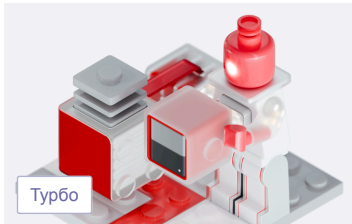
7

Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

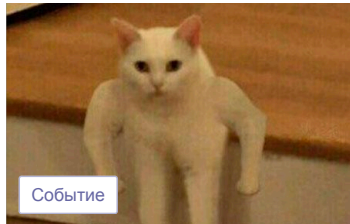
Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Собери облачный пазл и выиграй призы



Качаем к лету хард- и софт-скиллы на айтишных ивентах



Удалёнка в фарме: большие данные и никакой магии

ВОПРОСЫ И ОТВЕТЫ

Next.js App Router: Как правильно триггерить перезагрузку AdSense в SPA-layout при смене динамического параметра?

Next.js · Простой · 1 ответ

Как лучше реализовать тесты на проекте с symfony и nextjs?

Symfony · Простой · 1 ответ

Как сделать функцию "Поделиться фото в Вконтакте"?

ВКонтакте · Средний · 0 ответов

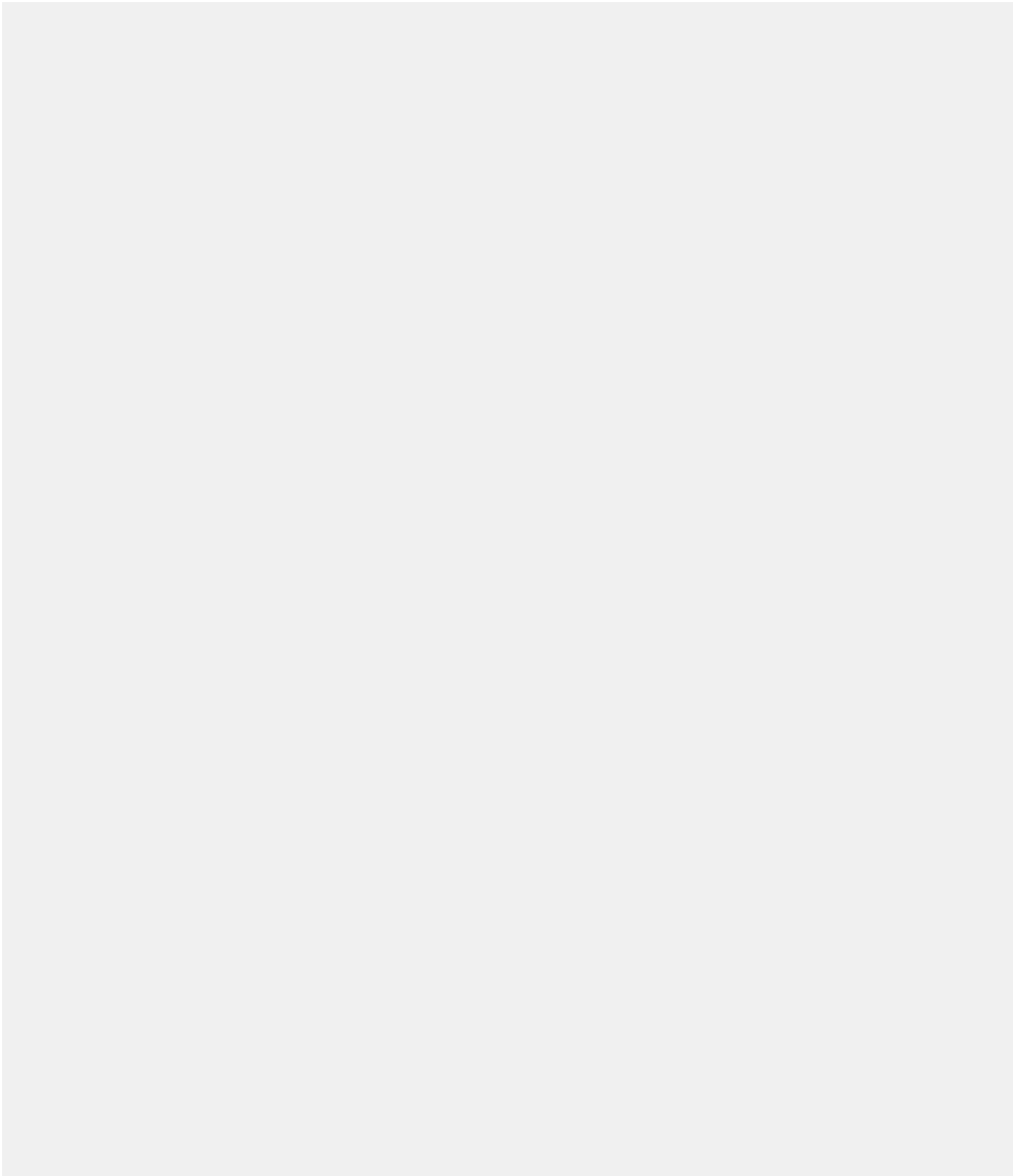
Как реализовать форму ввода как в веб-версии chatGPT?

HTML · Простой · 0 ответов

После деплоя на Vercel и привязки домена сайт не открывается с мобильных устройств?

DNS · Простой · 0 ответов

[Больше вопросов на Хабр Q&A](#)



ЧИТАЮТ СЕЙЧАС

Электроинструмент становится хуже, и это делается намеренно

 26K  81 

Тим Кук покидает пост Apple. Новый глава — «отец» Apple Silicon Джон Тернус

 13K  14

Представлен проект KillerPDF — редактор PDF с открытым исходным кодом для Windows 10/11

 6.7K  15

Пользователь купил Atomic Heart на VK Play, но не смог запустить игру из-за сетевых проблем

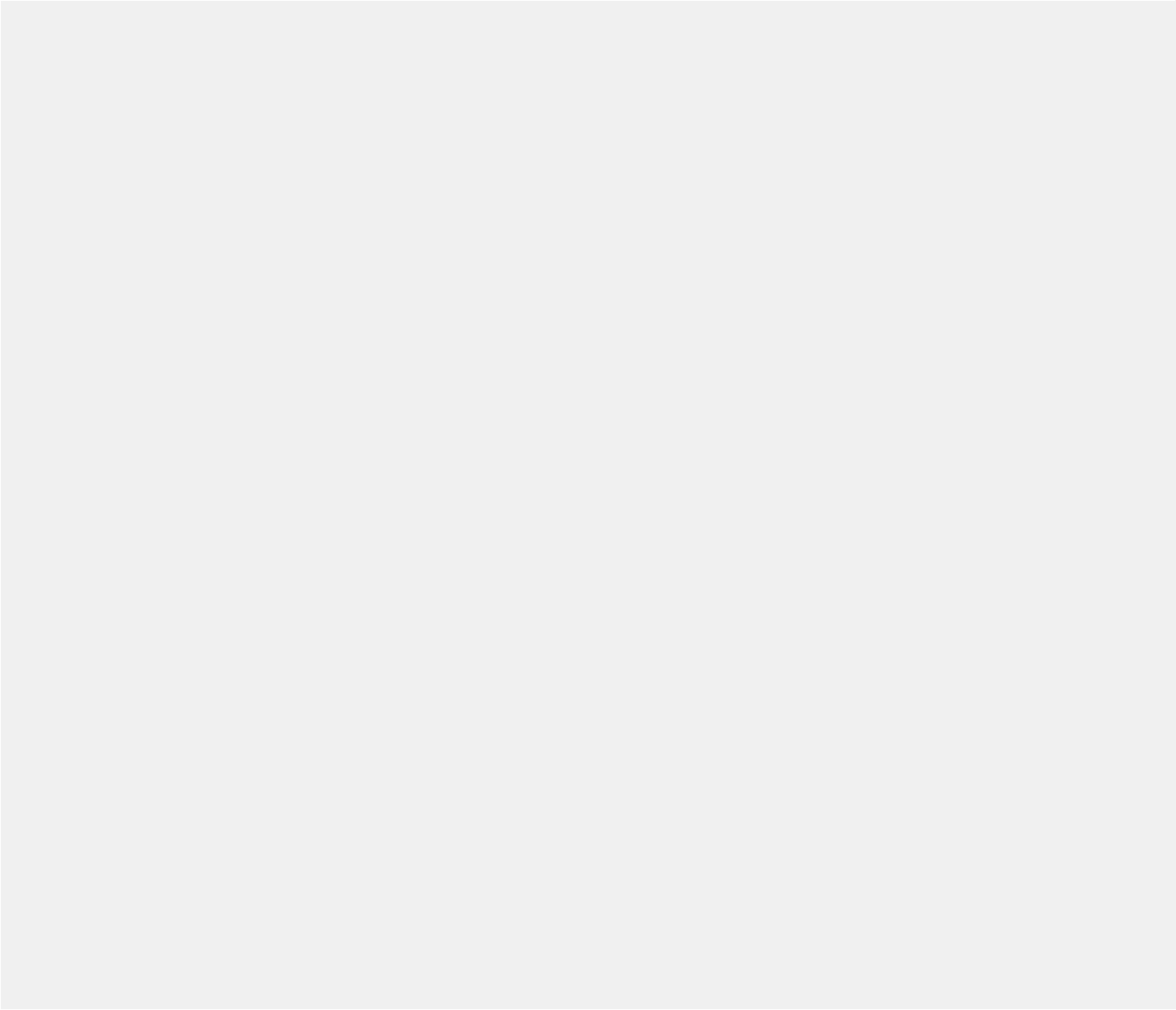
 5.3K  11

+185% за 13 часов: как Kimi K2.6 переписала 8-летний движок

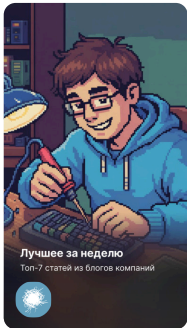
 14K  9

Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

Турбо



ИСТОРИИ



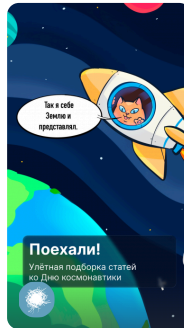
Годнота из блогов компаний



Только 10% решат эту головоломку



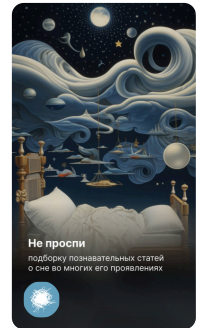
Хочу 450K



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать

Ваш аккаунт

[Войти](#)
[Регистрация](#)

Разделы

[Статьи](#)
[Новости](#)
[Хабы](#)
[Компании](#)
[Авторы](#)
[Песочница](#)

Информация

[Устройство сайта](#)
[Для авторов](#)
[Для компаний](#)
[Документы](#)
[Соглашение](#)
[Конфиденциальность](#)

Услуги

[Корпоративный блог](#)
[Медийная реклама](#)
[Нативные проекты](#)
[Образовательные программы](#)
[Стартапам](#)



[Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr